

DSA STUDY BLUEPRINT SERIES

Container with Most Water Blueprint

Two-Pointer Optimization and Area Bounds

Section 07 • C++ Core Data Structures & Algorithms

The Maximum Area Analysis

Finding two lines that contain the maximum water capacity

Problem Definition

- Given N heights forming vertical lines.
- Find two lines such that the container holds the most water.
- Area formula: $\text{Area} = \min(\text{height}[i], \text{height}[j]) * (j - i)$.
- Brute force calculates all combinations in $O(N^2)$ time.

Two-Pointer Intuition $O(N)$

- Place `left` at index 0 and `right` at index N-1.
- Width is maximized initially.
- To increase area, we must find taller lines.
- Rule: **Move the pointer pointing to the shorter height.**

Why Move the Shorter Pointer?

Mathematical proof of the greedy decision

The Logic

Let $\text{height}[\text{left}] < \text{height}[\text{right}]$. The area is bounded by $\text{height}[\text{left}]$.

- If we move the **right** pointer inward:
 - Width decreases.
 - Height cannot exceed $\text{height}[\text{left}]$ since area is limited by the minimum.
 - Thus, area can only **decrease** or remain same.
- If we move the **left** pointer inward:
 - Width decreases.
 - Height has a chance of becoming larger than $\text{height}[\text{left}]$, which may compensate for width reduction.

Two-Pointer Walkthrough

Heights: [1, 8, 6, 2, 5, 4, 8, 3, 7]

Left	Right	Width	Min Height	Area	Action
0 (H=1)	8 (H=7)	8	1	8	Move Left (1 < 7)
1 (H=8)	8 (H=7)	7	7	49 (Max)	Move Right (8 > 7)
1 (H=8)	7 (H=3)	6	3	18	Move Right (8 > 3)
1 (H=8)	6 (H=8)	5	8	40	Move Either

Two-Pointer Code in C++

Clean $O(N)$ Time and $O(1)$ Space implementation

```
#include
#include
#include
using namespace std;

int maxArea(vector& height) {
    int max_val = 0;
    int left = 0;
    int right = height.size() - 1;

    while (left < right) {
        int w = right - left;
        int h = min(height[left], height[right]);
        max_val = max(max_val, w * h);

        // Greedy step: Move the shorter line pointer
        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    return max_val;
}
```